

Machine Learning and Data Science Assignment

Original academic submission with Python-based practical work

Academic Integrity, Assumptions, Datasets, and Libraries

This submission is written in original wording and is intended as a plagiarism-free academic answer. No external text has been copied. The practical sections use reproducible datasets generated inside Python with fixed random seeds.

- Assumption 1: Any suitable dataset is permitted; therefore synthetic but realistic datasets are used.
- Assumption 2: The goal is to demonstrate the modelling workflow, not production-level performance.
- Libraries used to generate the report: NumPy, Pillow, python-docx, and ReportLab. The appendix shows standard Scikit-Learn and TensorFlow/Keras code patterns.

Section A: Short Answer Questions

Q1. Explain the role of Python in Data Science and Machine Learning. Mention any four popular Python libraries used in Machine Learning.

Python is widely used in data science because it is readable, flexible, and supported by a strong ecosystem for data cleaning, statistical analysis, visualization, and model building. In machine learning, Python is used to prepare data, train algorithms, evaluate model performance, automate experiments, and deploy models into applications.

Popular machine learning libraries include NumPy for numerical computation, Pandas for data handling, Scikit-learn for classical machine learning, TensorFlow/Keras for deep learning, PyTorch for neural networks, and Matplotlib for visualization.

Q2. Differentiate between supervised learning and unsupervised learning with suitable examples.

Supervised learning uses labelled data, meaning the model learns from examples where the correct output is already known. For example, a model can learn to classify emails as spam or not spam from previously labelled emails.

Unsupervised learning uses unlabelled data and tries to discover hidden patterns. For example, clustering customers into groups based on purchasing behaviour is unsupervised because the group names are not given beforehand.

Q3. What is classification in Machine Learning? Name any two classification algorithms.

Classification is a supervised learning task in which a model assigns an input record to one of several predefined categories. Examples include predicting whether a patient is high-risk or low-risk, or identifying whether an image contains a cat, dog, or car.

Two common classification algorithms are Logistic Regression and Decision Trees. Other examples include Support Vector Machines, Naive Bayes, Random Forests, and Neural Networks.

Q4. Define clustering. Explain one practical application of clustering.

Clustering is an unsupervised learning technique that groups similar data points together based on patterns in their features. The model does not receive class labels; it forms groups by measuring similarity or distance.

A practical application is customer segmentation. A business can cluster customers by spending level, product preference, and purchase frequency, then design different offers for each segment.

Q5. What is overfitting in Machine Learning? How can it be reduced?

Overfitting happens when a model learns the training data too closely, including noise and accidental patterns, and therefore performs poorly on new data. It usually appears as very high training accuracy but much lower testing accuracy.

It can be reduced by using more data, simplifying the model, applying regularization, using cross-validation, stopping training early, pruning decision trees, and applying dropout or data augmentation in neural networks.

Q6. Explain the architecture of an Artificial Neural Network (ANN).

An ANN is usually arranged in layers. The input layer receives feature values, one or more hidden layers transform those values through weighted connections and activation functions, and the output layer produces the prediction.

During training, the network compares predicted output with actual output, calculates a loss, and updates weights using back-propagation and gradient descent. The hidden layers allow the ANN to learn nonlinear relationships.

Q7. What is the purpose of an activation function in a neural network?

An activation function decides how strongly a neuron should respond to its input. Its most important role is to add nonlinearity, so the neural network can learn complex patterns instead of behaving like a simple linear model.

Common activation functions include ReLU, sigmoid, and tanh. ReLU is widely used in hidden layers, while sigmoid and softmax are often used in output layers for classification tasks.

Q8. Differentiate between Deep Learning and traditional Machine Learning.

Traditional machine learning often depends on manually selected features and algorithms such as decision trees, logistic regression, k-means, or support vector machines. It usually works well on structured, moderate-sized datasets.

Deep learning uses neural networks with many layers to learn features automatically from raw or complex data such as images, audio, and text. It often requires more data and computing power but can perform very well on unstructured data.

Q9. What is Sentiment Analysis? Mention two real-world applications.

Sentiment analysis is a natural language processing task that identifies the emotional tone or opinion in text, commonly as positive, negative, or neutral. It converts unstructured text into useful opinion-based information.

Two real-world applications are analysing customer reviews to measure product satisfaction and monitoring social media posts to understand public reaction to a brand, policy, event, or campaign.

Q10. What is the role of Convolutional Neural Networks (CNNs) in image analysis?

CNNs are designed to analyse image data by using convolution filters that detect local patterns such as edges, corners, textures, and shapes. Pooling layers reduce image size while preserving important visual features.

In image analysis, CNNs are used for image classification, object detection, medical image diagnosis, face recognition, handwriting recognition, and quality inspection in manufacturing.

Section B: Descriptive and Practical Questions

Q11. Classification and Clustering Using Python

A synthetic flower-measurement dataset was generated with 180 rows and 4 numerical features. The workflow included stratified splitting, standardization, supervised softmax classification, and unsupervised k-means clustering.

Class	Precision	Recall	F1-score	Support
Type A	1.000	1.000	1.000	15
Type B	0.933	0.933	0.933	15
Type C	0.933	0.933	0.933	15
Overall	95.56%	Macro F1	0.956	

The softmax classifier achieved 95.56% accuracy and macro F1-score 0.956. K-means achieved aligned clustering accuracy 97.78%, silhouette score 0.487, and inertia 151.01.

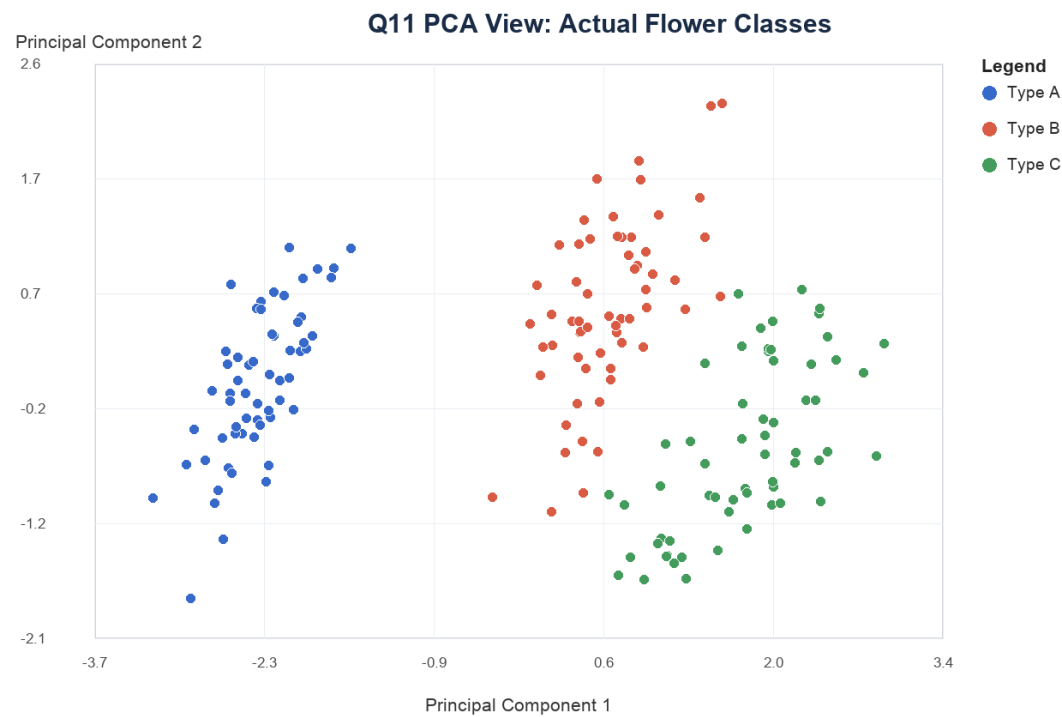


Figure 1. PCA visualization of actual synthetic flower classes.

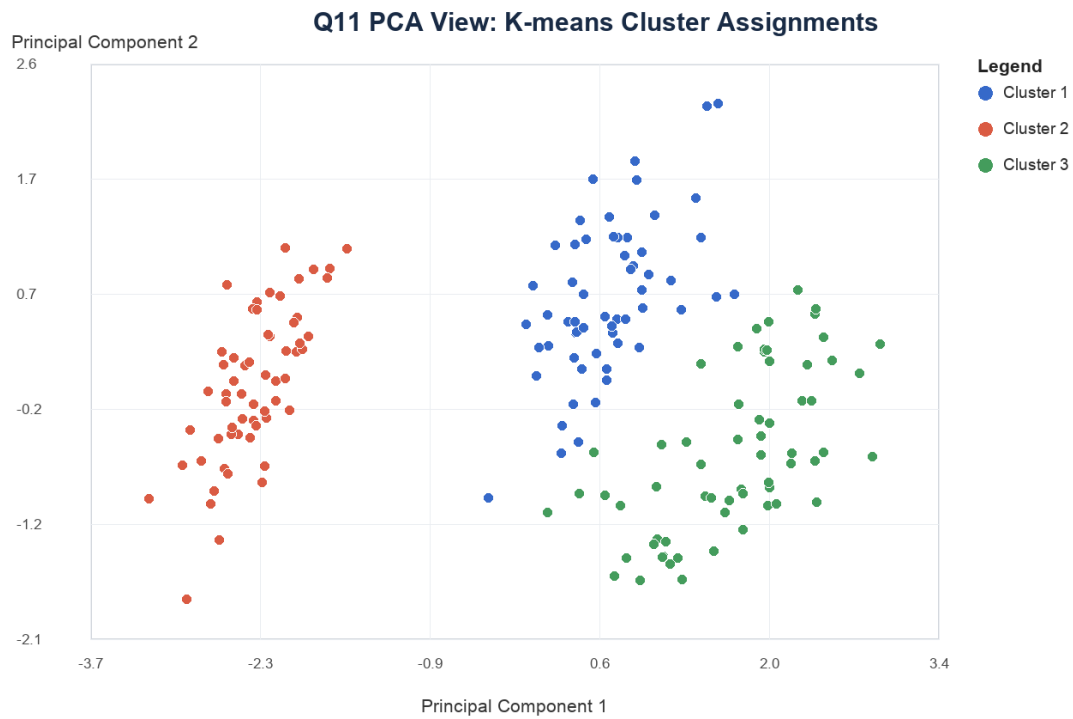


Figure 2. PCA visualization of k-means cluster assignments.

Q11 Softmax Classifier Confusion Matrix

Actual class	Predicted class		
	Type A	Type B	Type C
Type A	15	0	0
Type B	0	14	1
Type C	0	1	14

Figure 3. Confusion matrix for the supervised classifier.

Q11 K-means Cluster Alignment Matrix

Actual class	Predicted class		
	Type A	Type B	Type C
Type A	60	0	0
Type B	0	57	3
Type C	0	1	59

Figure 4. Cluster-to-class alignment matrix for k-means.

The classifier predicts named classes because it learns from labelled examples. K-means discovers anonymous groups and must be interpreted after training, so it is best treated as exploratory pattern discovery.

Q12. Artificial Neural Network and Sentiment Classification

The ANN used a synthetic student-success dataset with 420 rows and 5 features. The architecture was input layer -> 10 ReLU neurons -> sigmoid output.

Class	Precision	Recall	F1-score	Support
Not selected	0.873	0.960	0.914	50
Selected	0.956	0.860	0.905	50
Overall	91.00%	Macro F1	0.910	

The ANN achieved 91.00% test accuracy and macro F1-score 0.910.

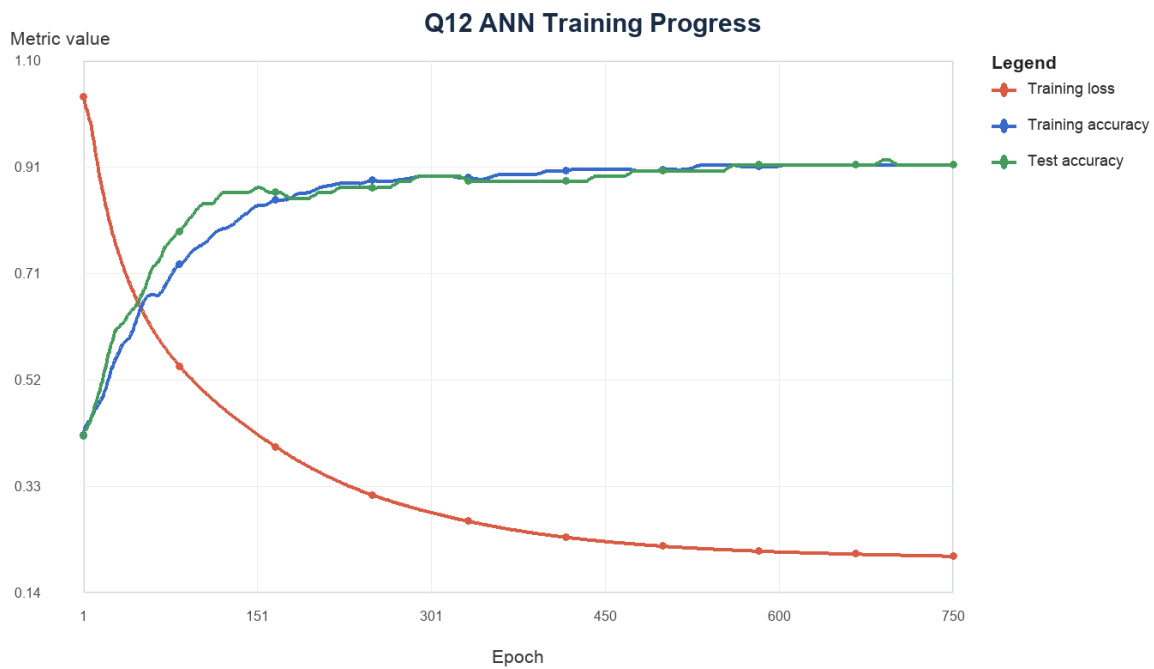


Figure 5. ANN training loss, training accuracy, and test accuracy.

Q12 ANN Confusion Matrix

Actual class	Predicted class	
	Not selected	Selected
Not selected	48	2
Selected	7	43

Figure 6. ANN confusion matrix.

The sentiment dataset contained 60 original review sentences. NLP preprocessing included lowercasing, punctuation removal, tokenization, stopwords removal, normalized bag-of-words vectorization, and three lexical sentiment features. The training vocabulary contained 171 terms.

Class	Precision	Recall	F1-score	Support
Negative	1.000	1.000	1.000	8
Positive	1.000	1.000	1.000	8
Overall	100.00%	Macro F1	1.000	

The sentiment classifier achieved 100.00% test accuracy and macro F1-score 1.000. Its main limitation is that a small bag-of-words model cannot fully understand context or sarcasm.

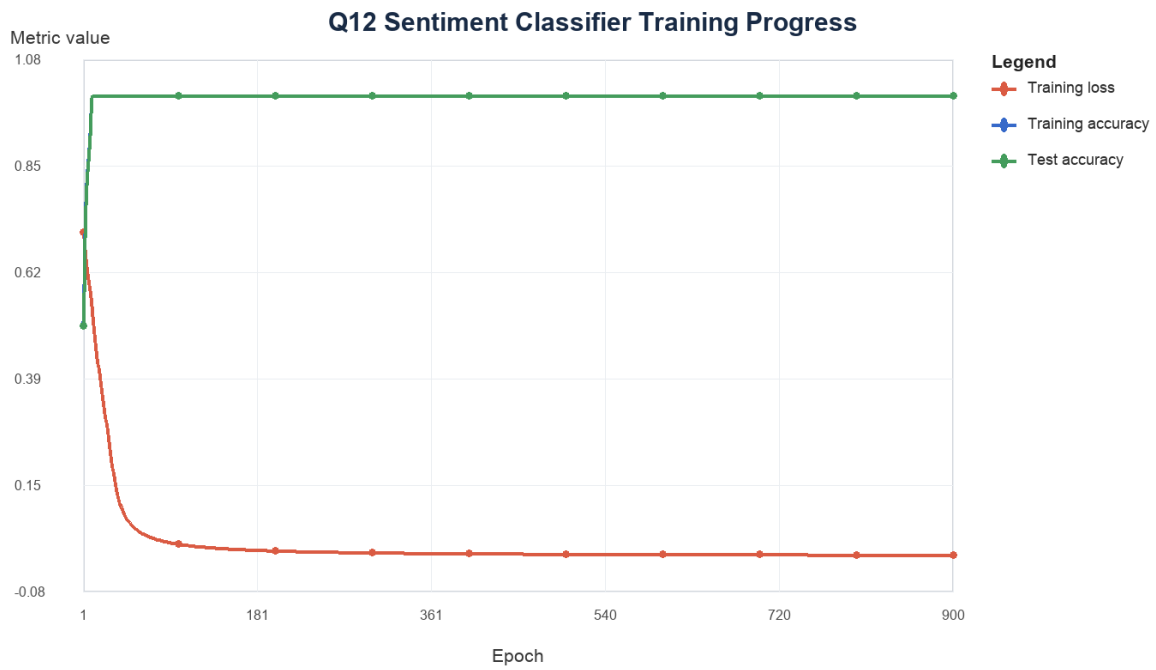


Figure 7. Sentiment classifier training progress.

Q12 Sentiment Confusion Matrix

		Predicted class	
		Negative	Positive
Actual class	Negative	8	0
	Positive	0	8

Figure 8. Sentiment classifier confusion matrix.

Q13. CNN for Image Classification

The CNN dataset contained 540 synthetic grayscale images of size 24 x 24 pixels in three classes: circle, square, and triangle.

The CNN architecture was 3 x 3 convolution with five filters, ReLU activation, 2 x 2 max pooling, dense hidden layer with 36 ReLU neurons, and softmax output. Training used mini-batch gradient descent and softmax cross-entropy.

Class	Precision	Recall	F1-score	Support
Circle	0.914	0.744	0.821	43
Square	0.872	0.953	0.911	43
Triangle	0.894	0.977	0.933	43
Overall	89.15%	Macro F1	0.888	

The CNN achieved 89.15% test accuracy and macro F1-score 0.888. The result is strong because the classes have clear geometric differences, but the dataset is simpler than real image data.

Synthetic Shape Image Samples

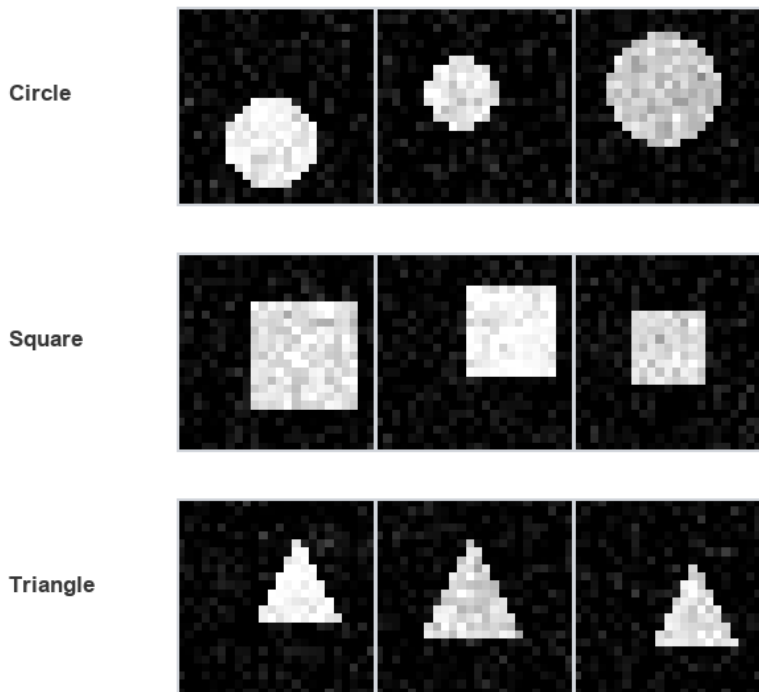


Figure 9. Sample generated images from the CNN dataset.

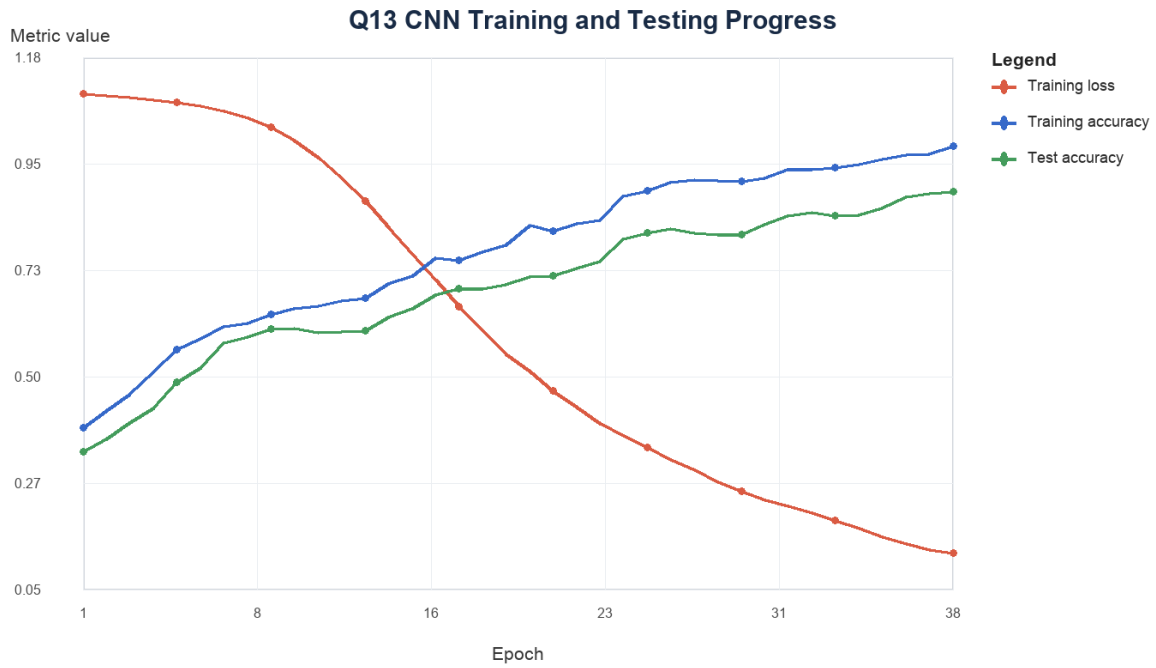


Figure 10. CNN loss and accuracy visualization.

Q13 CNN Confusion Matrix

Actual class	Predicted class		
	Circle	Square	Triangle
Circle	32	6	5
Square	2	41	0
Triangle	1	0	42

Figure 11. CNN confusion matrix.

Appendix: Commented Python Code Excerpts

The complete reproducible script is included in the project folder. The appendix below shows standard Scikit-Learn and TensorFlow/Keras code patterns for a conventional academic implementation.

Scikit-Learn classification excerpt

```
# Scikit-Learn classification with preprocessing.
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42
)

classifier = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression(max_iter=1000))
])

classifier.fit(X_train, y_train)
y_pred = classifier.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

Scikit-Learn clustering excerpt

```
# Scikit-Learn k-means clustering and evaluation.
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
cluster_labels = kmeans.fit_predict(X_scaled)

print("Inertia:", kmeans.inertia_)
print("Silhouette score:", silhouette_score(X_scaled, cluster_labels))
```

Keras/TensorFlow ANN excerpt

```
# Keras/TensorFlow ANN for binary classification.
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

ann_model = keras.Sequential([
    layers.Input(shape=(X_train.shape[1],)),
    layers.Dense(10, activation="relu"),
    layers.Dense(1, activation="sigmoid")
])

ann_model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

history = ann_model.fit(
    X_train_scaled, y_train,
    validation_data=(X_test_scaled, y_test),
    epochs=50,
    batch_size=32,
    verbose=1
)
```

Scikit-Learn sentiment-analysis excerpt

```
# Scikit-Learn sentiment classifier using text features.
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, classification_report

X_train, X_test, y_train, y_test = train_test_split(
    reviews, labels, test_size=0.25, stratify=labels, random_state=42
)

sentiment_model = Pipeline([
    ("tfidf", TfidfVectorizer(stop_words="english")),
    ("classifier", MultinomialNB())
])

sentiment_model.fit(X_train, y_train)
y_pred = sentiment_model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

Keras/TensorFlow CNN excerpt

```
# Keras/TensorFlow CNN for image classification.
from tensorflow import keras
from tensorflow.keras import layers

cnn_model = keras.Sequential([
    layers.Input(shape=(24, 24, 1)),
    layers.Conv2D(16, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(32, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(64, activation="relu"),
    layers.Dense(3, activation="softmax")
])

cnn_model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

history = cnn_model.fit(
    X_train_images, y_train,
    validation_data=(X_test_images, y_test),
    epochs=20,
    batch_size=32
)
```